



Sistemas operativos

Licenciatura em Engenharia Eletrónica e Telecomunicações e de Computadores

Trabalho prático 3

Grupo 66

50275 Dilan Martins

48199 Ruben Silva

2º Semestre letivo 2023/2024

08 de junho de 2024

Índice

LISTA DE FIGURAS	VI
1. PROGRAMAS DE PRODUÇÃO DE VALORES.....	1
2. PROGRAMA CHARGE POINT	3
3. PROGRAMA COUNT DOWN	9
4. PROGRAMAS THREAD POOL	11
5. PROGRAMA VECTOR WITH THREAD POOL.....	14
1. QUESTÕES DE ESCOLHA MÚLTIPLA.....	16

Lista de Figuras

Figura 1-Output do programa 1.a) antes da correção.....	1
Figura 2-Correção no código do programa 1.a).....	1
Figura 3 - Inicialização da estrutura para gestão de carregamento	3
Figura 4 - Estrutura chargepoint	4
Figura 5 - Função charge point destroy	4
Figura 6 - Função realizada por cada thread.....	4
Figura 7 - Função reserveChargePoint.....	5
Figura 8 - Função freeChargePoint.....	6
Figura 9 - Programa de teste de chargepoint.....	7
Figura 10 - Funções do programa countdown_t	9
Figura 11 - Função de trabalho do countdown_t	9
Figura 12 - main de teste do countdown_t.....	9
Figura 13 - Output do teste do programa countdown_t.....	10
Figura 14 - Estrutura da threadpool_t	11
Figura 15 - Função de trabalho da threadpool_t	11
Figura 16 - Função threadpool_init.....	11
Figura 17 - Função threadpool_submit	12
Figura 18 - Função threadpool_destroy	12
Figura 19 - Programa teste da threadpool_t.....	12
Figura 20 - Output do programa de teste da threadpool_t	13
Figura 21-Estrutura do item de trabalho a ser processado do vector_threadpool.....	14
Figura 22 - Função de processamento do item de trabalho do vector_threadpool	14
Figura 23 - Função vector_get_in_range com recurso a threadpool.....	15
Figura 24 - Função main do vector_threadpool	15

1. Programas de produção de valores

1.a) O programa não produz sempre o mesmo valor porque os argumentos passados a cada thread é uma variável através da qual vão realizar operações. Contudo como a ordem de execução das threads não é definida e ocorre aleatoriamente então o valor resultante das operações das várias threads é diferente consoante a ordem de execução das mesmas.

```
aluno@so-vm:~/Desktop/Trabalho 3$ make exerc1
gcc -g -Wall -c exerc1.c
gcc -g -Wall exerc1.o -o exerc1 -lm -pthread
aluno@so-vm:~/Desktop/Trabalho 3$ ./exerc1
Th3: 4043736
Th2: 2786853
Th1: 6920426
Total = 6920426
aluno@so-vm:~/Desktop/Trabalho 3$ ./exerc1
Th1: -743827
Th3: -4848209
Th2: -8481556
Total = -8481556
aluno@so-vm:~/Desktop/Trabalho 3$ ./exerc1
Th1: -706906
Th2: -3282704
Th3: 1272580
Total = 1272580
aluno@so-vm:~/Desktop/Trabalho 3$ ./exerc1
Th3: -2190095
Th1: -2504970
Th2: -2540223
Total = -2540223
```

Figura 1-Output do programa 1.a) antes da correção

A solução para a resolução desta irregularidade é através do emprego de variáveis diferentes variáveis aquando da criação de cada thread. Nesta situação é utilizado um array de três elementos para passar à thread o devido count e para efetuar a soma de todos os valores retornados pela thread.

```
38 int main()
39 {
40     int count = 0;
41
42     int arguments[3];
43     for(int i=0; i<3; i++)
44         arguments[i] = count;
45
46     pthread_t t1, t2, t3;
47     pthread_create(&t1, NULL, th1, &arguments[0]);
48     pthread_create(&t2, NULL, th2, &arguments[1]);
49     pthread_create(&t3, NULL, th3, &arguments[2]);
50     pthread_join(t1, NULL);
51     pthread_join(t2, NULL);
52     pthread_join(t3, NULL);
53
54     printf("---Results---\n");
55     for(int i=0; i<3; i++)
56     {
57         count = count + arguments[i];
58         printf("Argument %d, value %d\n", i, arguments[i]);
59     }
60     printf("Total = %d\n", count);
61 }
```

Figura 2-Correção no código do programa 1.a)

1.b) Á semelhança do código corrigido na alínea anterior o código apresentado por esta questão respeita os requisitos da não partilha mútua de variáveis entre threads diferentes, para não introduzir erros. Assim o código apresentado como opera variáveis distintas que são unicamente fundidas no fim da execução da threads, então, o mesmo apresenta sempre o mesmo valor de resultado.

2. Programa Charge Point

A implementação de um sistema de reserva de locais de carregamento deve ser faseado. O objetivo é a gestão de robôs nos pontos de carregamento tendo por base que só existem 10 destes pontos e que cada robô sabe exatamente a posição livre a que se deve dirigir para recarregar as suas baterias. É criado também o suporte para robôs prioritários que atendendo ao seu serviço devem conseguir um local de carregamento o mais rápido possível.

```
// Inicialização dos pontos de carregamento
void chargepoint_init(chargepoint* chargepoint)
{
    // Atualização das variáveis de apoio a zero
    for (int i = 0; i < MAX_CHARGEPOINT; i++) {
        chargepoint->position[i] = -1;
        chargepoint->id[i] = 0;
    }
    chargepoint->nrSpots = 0;
    chargepoint->count = 0;
    chargepoint->waitingThreads = 0;
    chargepoint->priorityWait = 0;

    // Inicialização dos mecanismos de sincronismo
    pthread_mutex_init(&(chargepoint->lock), NULL);
    pthread_cond_init(&(chargepoint->wait), NULL);
    pthread_cond_init(&(chargepoint->priorityWaitCond), NULL);
}
```

Figura 3 - Inicialização da estrutura para gestão de carregamento

A função `chargepoint_init` recebe o ponteiro para a estrutura `chargepoint` preparando a mesma para uso. Essa estrutura é responsável por guardar as posições dos pontos de carregamento, os ID's das threads que simulam os robôs a carregar, o número de pontos de carregamento e a contagem dos carregadores ocupados. Além disso a estrutura faz uso de um bloqueio de exclusão múltipla `mutex` para controlo de acesso a recursos partilhados entre threads e um mecanismo de sincronização para a gestão de múltiplas threads.


```
typedef struct {
    int position[MAX_CHARGEPOINT];
    long int id[MAX_CHARGEPOINT];
    int nrSpots;
    int count;
    int waitingThreads;
    int priorityWait;
    pthread_mutex_t lock;
    pthread_cond_t wait;
    pthread_cond_t priorityWaitCond;
} Chargepoint;
```

Figura 4 - Estrutura chargepoint

A função chargepoint_destroy é responsável por acabar com o ciclo de carregamento.

```
// Fim dos pontos de carregamento
void chargepoint_destroy(Chargepoint* chargepoint)
{
    // Finalização dos mecanismos de sincronização
    pthread_mutex_destroy(&(chargepoint->lock));
    pthread_cond_destroy(&(chargepoint->wait));
    pthread_cond_destroy(&(chargepoint->priorityWaitCond));
}
```

Figura 5 - Função charge point destroy

Cada robot inserido no sistema é simulado através do uso de uma thread.

Cada thread gera um tempo de espera aleatório que corresponde ao tempo em produção de um robô, após esse tempo é necessário o estacionamento no carregador para recarregar as baterias. A prioridade da thread é gerada também para a escolha da função que irá utilizar.

```
void* thread_func(void * chargepoint)
{
    Chargepoint * point = (Chargepoint*) chargepoint;
    int secs = rand() % 5;
    pthread_t thId = pthread_self();
    int priority = rand() % 2;

    printf("[%ld] - Sleeping for %d secs - Priority(%d)\n", thId, secs, priority);
    sleep(secs);

    int ticket;

    if(priority == 0)
        ticket = reserveChargePoint(point);
    else
        ticket = reserveChargePointPriority(point);

    printf("[%ld] - Parking for %d secs in postion (%d|%d) - Priority(%d)\n", thId, secs, ticket, MAX_CHARGEPOINT, priority);
    sleep(secs);
    printf("[%ld] - Unparked in postion (%d|%d) - Priority(%d)\n", thId, ticket, MAX_CHARGEPOINT, priority);
    freeChargePoint(point);

    return NULL;
}
```

Figura 6 - Função realizada por cada thread

A gestão de estacionamento no carregador é efetuada através da função `reserveChargePoint` que gere diversos aspetos seguindo o procedimento:

- Uso de mutex para garantir o acesso exclusivo;
- Procura de uma posição de carregamento livre;
- Se encontrar uma posição livre atualiza o array posição e insere no array ID o id da thread que fez a reserva;
- Imprime o estado dos pontos de carregamento;
- Liberta o mutex para permitir o acesso de outras threads.

```
// Reserva ponto de carregamento e devolve identificador do ponto de carga
int reserveChargePoint(chargepoint *chargepoint)
{
    // Bloqueio do acesso ao recurso crítico
    pthread_mutex_lock(&(chargepoint->lock));
    int myPosition = -1;
    chargepoint->waitingThreads++;

    do {
        // Verificação de uma posição de carregamento livre
        for (int i = 0; i < MAX_CHARGEPOINT; i++)
        {
            if (chargepoint->position[i] == -1)
            {
                myPosition = i;
                break;
            }
        }

        // Se uma posição livre for encontrada
        if (myPosition != -1)
        {
            // Se houver threads prioritárias espera
            if (chargepoint->priorityWait > 0)
            {
                pthread_cond_wait(&(chargepoint->priorityWaitCond), &(chargepoint->lock));
                myPosition = -1; // Reset à posição
            }
            else
            {
                break;
            }
        }
        else {
            // Espera passiva das threads normais
            pthread_cond_wait(&(chargepoint->wait), &(chargepoint->lock));
        }
    } while (1);

    chargepoint->waitingThreads--;

    // Atualiza a posição de carregamento
    chargepoint->position[myPosition] = myPosition;
    chargepoint->id[myPosition] = pthread_self();

    printf("\nAcesso a recurso crítico thread nº:%d com ID:%ld\n", chargepoint->count++, pthread_self());
    for (int i = 0; i < MAX_CHARGEPOINT; i++) {
        printf("\t %d \t %ld\n", chargepoint->position[i], chargepoint->id[i]);
    }

    // Recursos críticos
    chargepoint->nrSpots++;

    // Desbloqueio do acesso ao recurso crítico
    pthread_mutex_unlock(&(chargepoint->lock));

    return myPosition;
}
```

Figura 7 - Função `reserveChargePoint`

O procedimento fundamental desta função assenta no uso do mutex e da condition variable. O mutex concede a exclusividade de acesso a apenas uma thread com relação a um recurso que deve ser partilhado entre todas as threads, nesta situação o mutex garante que apenas uma thread de cada vez faz uso da reserva do ponto de carregamento. A condition variable é utilizada para verificar quais as posições livres de carregamento e sempre que uma seja encontrada para que a thread que a encontrou fique com o lugar de carregamento. Se nenhuma posição estiver livre todas as threads ficam em espera e farão um novo teste apenas com após a libertação de um ponto de carregamento que é coordenada pela função freeChargePoint. A função possui ainda a capacidade de apenas ocupar o lugar de carregamento que encontrou vago apenas se não existir nenhuma thread prioritária em espera.

```
// Liberta ponto de carregamento
void freeChargePoint(chargepoint *chargepoint)
{
    // Bloqueamento do acesso ao recurso crítico
    pthread_mutex_lock(&(chargepoint->lock));

    // Obtenção do ID da thread que acabou
    long int id = pthread_self();

    // Libertação da posição do identificador de carregamento
    for (int i = 0; i < MAX_CHARGEPOINT; i++) {
        if (chargepoint->id[i] == id) {
            chargepoint->position[i] = -1;
            chargepoint->id[i] = 0;
            break;
        }
    }

    // Recursos críticos
    chargepoint->nrSpots--;

    // Ativação de threads em espera
    if (chargepoint->priorityWait > 0) {
        pthread_cond_signal(&(chargepoint->priorityWaitCond));
    } else {
        pthread_cond_signal(&(chargepoint->wait));
    }

    // Desbloqueio do acesso ao recurso crítico
    pthread_mutex_unlock(&(chargepoint->lock));
}
```

Figura 8 - Função freeChargePoint

A função freechargepoint usa o bloqueio mutex para que a libertação de uma posição de carregamento de cada vez, permitindo o uso da função de uma única thread. Através do ID da thread que executa a função, procura no array o seu lugar de carregamento e desassocia o ID do array e da posição. Atualiza o número de espaços ocupados e acorda a próxima thread que estava à espera de uma posição de carregamento. Por fim desbloqueia o acesso a esta função para que outras threads a possam usar.

```

Acesso a recurso critico thread nº:29 com ID:140391860819520
0      140391953139264
1      140392214861376
2      140392053851712
3      140391894390336
4      140391877604928
5      140391860819520
-1     0
7      140391844034112
8      140391793677888
9      140391835641408
[140391860819520] - Parking for 2 secs in postion (5|10)
[140391844034112] - Unparked          in postion (7|10)
[140391793677888] - Unparked          in postion (8|10)
[140391835641408] - Unparked          in postion (9|10)

Acesso a recurso critico thread nº:30 com ID:140391827248704
0      140391953139264
1      140392214861376
2      140392053851712
3      140391894390336
4      140391877604928
5      140391860819520
6      140391827248704
-1     0
-1     0
-1     0
[140391827248704] - Parking for 2 secs in postion (6|10)

```

Figura 9 - Programa de teste de chargepoint

A figura anterior demonstra a aplicação prática das funções anteriormente explicadas.

A linha de acesso ao recurso crítico nº29 representa o ponto onde a thread (520) entra para o lugar de carregamento 5, esse lugar é registado com o Id da thread e a mensagem de parking seguinte confirma esse procedimento. Quando as threads saem do lugar de carregamento é mostrada a mensagem unparked e retirado o ID do lugar correspondente. Assim podem sucessivamente assumir threads os lugares livres, sendo que as mesmas entram sempre no menor número identificador de um lugar de carregamento livre.

```

// Reserva ponto de carregamento com prioridade e devolve identificador do ponto de carga
int reserveChargePointPriority(chargepoint *chargepoint)
{
    // Bloqueio do acesso ao recurso crítico
    pthread_mutex_lock(&(chargepoint->lock));

    int myPosition = -1;
    chargepoint->waitingThreads++;
    chargepoint->priorityWait++;

    do {
        // Verificação de uma posição de carregamento livre
        for (int i = 0; i < MAX_CHARGEPOINT; i++)
        {
            if (chargepoint->position[i] == -1)
            {
                myPosition = i;
                break;
            }
        }

        // Se uma posição livre for encontrada
        if (myPosition != -1)
        {
            break;
        }

        pthread_cond_wait(&(chargepoint->priorityWaitCond), &(chargepoint->lock));
    } while (1);

    chargepoint->waitingThreads--;
    chargepoint->priorityWait--;

    // Atualiza a posição de carregamento
    chargepoint->position[myPosition] = myPosition;
    chargepoint->id[myPosition] = pthread_self();

    printf("\nAcesso a recurso critico thread nº:%d com ID:%ld com prioridade\n", chargepoint->count++, pthread_self());
    for (int i = 0; i < MAX_CHARGEPOINT; i++) {
        printf("\t %d \t %ld\n", chargepoint->position[i], chargepoint->id[i]);
    }

    // Recursos críticos
    chargepoint->nrSpots++;

    // Desbloqueio do acesso ao recurso crítico
    pthread_mutex_unlock(&(chargepoint->lock));

    return myPosition;
}

```

Figura 10 - Programa prioritário

A função `reservechargepointpriority` faz a reserva de um local de carregamento prioritário, a mesmo identifica o número de threads prioritárias que estão em espera para condicionar a aceitação de vagas de threads não prioritárias. Ao trabalhar em conjunto com a função `reservechargepoint` é garantida a prioridade de threads quando as mesmas existem.

3. Programa countdown_t

O programa `countdown_t` apresentado tenta dar resposta ao pedido no enunciado, sendo este um programa de sincronismo em que uma ou várias tarefas esperam pelas subseqüentes até o contador ser zero. Quando o contador chegar a zero, todas as threads que esperaram são desbloqueadas e retornam ao `main`, todas as threads subseqüentes, antes sequer de chamarem o `countdown_wait()`, verificam que o contador está a zero e terminam sem retornar qualquer valor.

```
typedef struct {
    int value;
    pthread_mutex_t lock;
    pthread_cond_t wait;
} countdown_t;

int countdown_init (countdown_t *cd, int initialValue){
    cd->value = initialValue;
    pthread_mutex_init(&(cd->lock), NULL);
    pthread_cond_init(&(cd->wait), NULL);
    return 1;
}

int countdown_destroy (countdown_t *cd){
    pthread_mutex_destroy(&(cd->lock));
    pthread_cond_destroy(&(cd->wait));
    return 1;
}

int countdown_wait (countdown_t *cd){
    pthread_mutex_lock(&(cd->lock));
    while(cd->value > 0){
        pthread_cond_wait(&(cd->wait), &(cd->lock));
    }
    pthread_mutex_unlock(&(cd->lock));
    return 1;
}

int countdown_down (countdown_t *cd){
    pthread_mutex_lock(&(cd->lock));
    cd->value--;
    pthread_cond_broadcast(&(cd->wait));
    pthread_mutex_unlock(&(cd->lock));
    return cd->value;
}
```

Figura 11 - Funções do programa `countdown_t`

```
void* threadWork(void *arg){
    countdown_t * cd = (countdown_t *)arg;
    pthread_t id = pthread_self();
    int sec=8;
    printf("Thread %d is working for %d seconds\n", (int)id, sec );
    sleep(sec);
    if(countdown_down(cd)!=cd->value){
        printf("Error on countdown_t\n");
    }
    if(cd->value>=0){
        printf("Thread %d: Waiting all threads to finish\n", (int)id);
        countdown_wait(cd);
    }else{
        printf("Thread %d: ERROR: Out of scope of countdown_t, ending following threads\n", (int)id);
        pthread_exit(NULL);
    }
    printf("Thread %d: Finnished \n", (int)id);
    return NULL;
}
```

Figura 12 - Função de trabalho do `countdown_t`

```
int main(){
    pthread_t ths[MAX_THREADS];
    countdown_t cd;
    if(countdown_init(&cd, COUNTDOWN)!=1){
        printf("Error initializng countdown_t\n");
        return -1;
    };
    for(int i=0; i<MAX_THREADS; i++){
        pthread_create(&ths[i], NULL, threadWork, &cd);
        sleep(1);
    }
    for(int i=0; i<MAX_THREADS; i++){
        pthread_join(ths[i], NULL);
    }
    printf("-----\nAll Threads Finished\n-----\n");
    if(countdown_destroy(&cd)==0){
        return 0;
    }
    return -1;
}
```

Figura 13 - `main` de teste do `countdown_t`

Este código então produz o seguinte output:

```
aluno@so-vm:~/Desktop/Trabalho 3$ ./ex3
Thread -302647744 is working for 8 seconds
Thread -311040448 is working for 8 seconds
Thread -319433152 is working for 8 seconds
Thread -402655680 is working for 8 seconds
Thread -411048384 is working for 8 seconds
Thread -419441088 is working for 8 seconds
Thread -427833792 is working for 8 seconds
Thread -436226496 is working for 8 seconds
Thread -302647744: Waiting all threads to finish
Thread -311040448: Waiting all threads to finish
Thread -319433152: Waiting all threads to finish
Thread -402655680: Waiting all threads to finish
Thread -402655680: Finnished
Thread -311040448: Finnished
Thread -302647744: Finnished
Thread -319433152: Finnished
Thread -411048384: ERROR: Out of scope of countdown_t, ending following threads
Thread -419441088: ERROR: Out of scope of countdown_t, ending following threads
Thread -427833792: ERROR: Out of scope of countdown_t, ending following threads
Thread -436226496: ERROR: Out of scope of countdown_t, ending following threads
-----
All Threads Finished
-----
aluno@so-vm:~/Desktop/Trabalho 3$
```

Figura 14 - Output do teste do programa countdown_t

4. Programa threadpool_t

O programa threadpool_t, é um programa baseado numa arquitetura de produtor/consumidor que visa colocar as tarefas num buffer e a partir daí as threads irem esperando e eventualmente realizando essas tarefas, até o buffer ficar vazio ou até o programa ser encerrado. Ao encerrar a threadpool, esta ativa a flag stop, ocorrendo o bloqueio de inserção de work_items no buffer e espera que não hajam mais threads ativas. De forma a atender os requisitos do enunciado, realizou-se criou-se uma estrutura threadpool_t sem dependências de outras estruturas anteriormente criadas.

```
typedef void (*wi_function_t)(void *);

typedef struct {
    wi_function_t func;
    void *arg;
} work_item_t;

typedef struct {
    pthread_mutex_t lock;
    pthread_cond_t cond_var;
    pthread_cond_t cond_complete;
    pthread_t *ths;
    work_item_t *queue;
    int size;
    int count;
    int head;
    int tail;
    int nthreads;
    bool stop;
    int active_ths;
} threadpool_t;
```

Figura 15 - Estrutura da threadpool_t

```
// Função que será executada por cada thread do pool
void *worker_thread(void *arg) {
    threadpool_t *tp = (threadpool_t *)arg;
    while (true) {
        pthread_mutex_lock(&tp->lock);

        while (tp->count == 0 && !tp->stop) {
            pthread_cond_wait(&tp->cond_var, &tp->lock);
        }

        if (tp->stop && tp->count == 0) {
            pthread_mutex_unlock(&tp->lock);
            break;
        }

        work_item_t work = tp->queue[tp->head];
        tp->head = (tp->head + 1) % tp->size;
        tp->count--;

        tp->active_ths++;

        pthread_mutex_unlock(&tp->lock);

        work.func(work.arg);

        pthread_mutex_lock(&tp->lock);
        tp->active_ths--;
        if (tp->stop && tp->count == 0 && tp->active_ths == 0) {
            pthread_cond_signal(&tp->cond_complete);
        }
        pthread_mutex_unlock(&tp->lock);
    }
    return NULL;
}
```

Figura 16 - Função de trabalho da threadpool_t

```
// Inicializa o thread pool
int threadpool_init(threadpool_t *tp, int queueDim, int nthreads) {
    tp->size = queueDim;
    tp->count = 0;
    tp->head = 0;
    tp->tail = 0;
    tp->nthreads = nthreads;
    tp->stop = false;
    tp->active_ths = 0;
    tp->ths = (pthread_t *)malloc(nthreads * sizeof(pthread_t));
    tp->queue = (work_item_t *)malloc(queueDim * sizeof(work_item_t));
    pthread_mutex_init(&tp->lock, NULL);
    pthread_cond_init(&tp->cond_var, NULL);
    pthread_cond_init(&tp->cond_complete, NULL);

    for (int i = 0; i < nthreads; i++) {
        pthread_create(&tp->ths[i], NULL, worker_thread, tp);
    }

    return 0;
}
```

Figura 17 - Função threadpool_init


```

// Submete um trabalho ao thread pool
int threadpool_submit(threadpool_t *tp, wi_function_t func, void *args) {
    pthread_mutex_lock(&tp->lock);

    // Verifica se a fila está cheia
    if (tp->count == tp->size) {
        pthread_mutex_unlock(&tp->lock);
        return -1; // Fila cheia
    }

    // Adiciona o trabalho à fila
    tp->queue[tp->tail].func = func;
    tp->queue[tp->tail].arg = args;
    tp->tail = (tp->tail + 1) % tp->size;
    tp->count++;

    pthread_cond_signal(&tp->cond_var);
    pthread_mutex_unlock(&tp->lock);

    return 0;
}

```

Figura 18 - Função threadpool_submit

```

// Destroi o thread pool
int threadpool_destroy(threadpool_t *tp) {
    pthread_mutex_lock(&tp->lock);
    tp->stop = true;
    pthread_cond_broadcast(&tp->cond_var);

    while (tp->count > 0 || tp->active_ths > 0) {
        pthread_cond_wait(&tp->cond_complete, &tp->lock);
    }
    pthread_mutex_unlock(&tp->lock);

    for (int i = 0; i < tp->nthreads; i++) {
        printf("%ld exiting\n", pthread_self());
        pthread_join(tp->ths[i], NULL);
    }

    free(tp->ths);
    free(tp->queue);

    pthread_mutex_destroy(&tp->lock);
    pthread_cond_destroy(&tp->cond_var);
    pthread_cond_destroy(&tp->cond_complete);

    return 0;
}

```

Figura 19 - Função threadpool_destroy

Para testar este programa realizou-se um programa tptest que serviria apenas para testar a threadpool anterior:

```

#define NR_OF_THREADS (3)
#define MAX_ARGS (10)

void *ft(void * arg)
{
    int id = (int) arg;
    printf("%d enter on thread %ld\n", id, pthread_self());
    sleep(2);
    printf("%d exit on thread %ld\n", id, pthread_self());
}

int main()
{
    threadpool_t tp;
    threadpool_init(&tp, MAX_ARGS, NR_OF_THREADS);

    for(int i=0; i<MAX_ARGS; i++){
        if(i==5){threadpool_destroy(&tp);}
        threadpool_submit(&tp, ft, (void*)i);
        sleep(1);
    }
}

```

Figura 20 - Programa teste da threadpool_t

Este programa teve o seguinte output:

```
aluno@so-vm:~/Desktop/Trabalho 3$ ./thread_pool
0 enter on thread 140547058697792
1 enter on thread 140547050305088
0 exit on thread 140547058697792
2 enter on thread 140547041912384
1 exit on thread 140547050305088
3 enter on thread 140547058697792
4 enter on thread 140547050305088
2 exit on thread 140547041912384
3 exit on thread 140547058697792
4 exit on thread 140547050305088
140547058702144 exiting
140547058702144 exiting
140547058702144 exiting
aluno@so-vm:~/Desktop/Trabalho 3$
```

Figura 21 - Output do programa de teste da threadpool_t

Como se verifica apesar do buffer poder atender até 10 argumentos, quando chega ao quinto argumento a threadpool é sinalizada para terminar, não aceitando mais nenhum argumento, esperando que a última tarefa seja terminada e só após isso terminando o programa.

5. Programa vector_threadpool

À semelhança dos trabalhos anteriores, este exercício visa decompor um vetor de dimensão elevada em fatias de dimensão idêntica e essas fatias serem processadas pelos respectivos processos/threads, mas neste trabalho, será colocado processado por uma threadpool, que em teoria é muito mais eficiente, em termos de recursos computacionais. O programa então, baseado nos programas countdown_t e threadpool_t anteriores fará proposto da seguinte forma: O vetor é iniciado com valores aleatórios ou incrementais, será criada uma estrutura para guardar os argumentos de trabalho da work_function e será criada a própria work_function que irá processar então o seu troço do vetor. De forma a evitar deadlocks e conflitos de acesso, usou-se os instrumentos de sincronismo implementados nos programas countdown_t e threadpool_t bem como mais um que visa impedir ao máximo a concorrência ao argumento sv_sz fazendo com que este não seja maior do que o esperado.

```
typedef struct {
    int *v;
    int start;
    int end;
    int min;
    int max;
    int *sv;
    int *sv_sz;
    countdown_t *cd;
    pthread_mutex_t *sv_lock;
} work_item_args_t;
```

Figura 22-Estrutura do item de trabalho a ser processado do vector_threadpool

```
void* work_function(void *arg) {
    work_item_args_t *args = (work_item_args_t *)arg;
    int *v = args->v;
    int start = args->start;
    int end = args->end;
    int min = args->min;
    int max = args->max;
    int *sv = args->sv;
    int *sv_sz = args->sv_sz;
    pthread_mutex_t *sv_lock = args->sv_lock;

    for (int i = start; i < end; i++) {
        if (v[i] >= min && v[i] <= max) {
            pthread_mutex_lock(sv_lock);
            sv[*sv_sz] = v[i];
            (*sv_sz)++;
            pthread_mutex_unlock(sv_lock);
        }
    }

    countdown_down(args->cd);
    free(arg);
    return NULL;
}
```

Figura 23 - Função de processamento do item de trabalho do vector_threadpool

```

int vector_get_in_range_with_thread_pool(int v[], int v_sz, int sv[], int min, int max, threadpool_t *tp) {
    int nthreads = tp->nthreads;
    int chunk_size = (v_sz + nthreads - 1) / nthreads;
    int sv_sz = 0;
    pthread_mutex_t sv_lock;
    pthread_mutex_init(&sv_lock, NULL);

    countdown_t cd;
    countdown_init(&cd, nthreads);

    for (int i = 0; i < nthreads; i++) {
        int start = i * chunk_size;
        int end = (start + chunk_size > v_sz) ? v_sz : start + chunk_size;

        if (start >= v_sz) break;

        work_item_args_t *args = (work_item_args_t *)malloc(sizeof(work_item_args_t));
        args->v = v;
        args->start = start;
        args->end = end;
        args->min = min;
        args->max = max;
        args->sv = sv;
        args->sv_sz = &sv_sz;
        args->cd = &cd;
        args->sv_lock = &sv_lock;

        threadpool_submit(tp, work_function, (void *)args);
    }

    countdown_wait(&cd);

    countdown_destroy(&cd);
    pthread_mutex_destroy(&sv_lock);

    return sv_sz;
}

```

Figura 24 - Função vector_get_in_range com recurso a threadpool

```

int main() {
    int v[V_SZ];
    int v_sz = V_SZ;
    int min = MIN_VAL / 2;
    int max = MAX_VAL / 2;
    int sv[V_SZ]; // Tamanho máximo do subvetor é o mesmo que o vetor original
    int sv_sz;
    vector_get_incremental(v, v_sz);

    threadpool_t tp;
    int nthreads = N_THS;
    int queue_size = BUFF;

    threadpool_init(&tp, queue_size, nthreads);

    sv_sz = vector_get_in_range_with_thread_pool(v, v_sz, sv, min, max, &tp);

    threadpool_destroy(&tp);

    printf("Subvector with %d elements in range [%d, %d]:\n", sv_sz, min, max);
    for (int i = 0; i < sv_sz; i++) {
        printf("%d ", sv[i]);
    }
    printf("\n");

    return 0;
}

```

Figura 25 - Função main do vector_threadpool

6. Questões de escolha múltipla

1. Indique, para cada uma das afirmações, se a utilização de múltiplas tarefas, num programa a executar-se num sistema operativo UNIX, é uma razão válida.
 - Para ter a execução de dois troços de código concorrentes com espaços de endereçamento separados num mesmo processo. **INVÁLIDO** (em concorrência, os espaços de endereçamento são compartilhados)
 - Para poder realizar operações I/O em simultâneo com outras operações num mesmo processo. **VÁLIDO** (se as operações não forem ambas de escrita)
 - Para poder executar dois programas (ficheiros executáveis) diferentes em concorrência e de uma forma mais rápida. **INVÁLIDO** (a execução de programas diferentes requer o uso de espaço de endereçamentos diferentes, para tal é necessário o uso de múltiplos processos)
 - Para dividir o processamento por ações concorrentes de forma a maximizar a utilização de toda a capacidade de processamento do hardware. **VÁLIDO**
2. Considere que a função `handleClient()` processa uma ligação com o cliente indique se as afirmações seguintes são verdadeiras ou falsas.
 - Servidor disponível através de um socket no domínio internet atendendo múltiplos clientes em concorrência. **F**
 - Servidor disponível através de um socket no domínio internet atendendo múltiplos clientes em sequência. **V**
 - Servidor disponível através de um socket no domínio internet atendendo múltiplos clientes em concorrência. **F**
 - Servidor disponível através de um socket no domínio internet atendendo múltiplos clientes em sequência. **V**